

PAPanner: Progress-Aware Planner for Embodiment-Agnostic Vision-and-Language Navigation

Anonymous TASE submission

Abstract—Vision-and-language navigation in continuous environments (VLN-CE) has achieved remarkable progress with the integration of vision-language models (VLMs). However, when following long-horizon instructions, VLM-based navigation faces a key challenge: the reliance on implicit temporal reasoning often leads to unreliable estimation of instruction execution progress. To address this issue, we propose PAPanner, a progress-aware planning framework for VLN-CE. PAPanner first formulates navigation as a progress-aware planning problem by decomposing each instruction into ordered sub-instructions. Then, the VLM explicitly tracks the execution progress while predicting navigation commands within a structured action space, which significantly enhances the reasoning stability. Finally, PAPanner executes such commands through a locomotion policy that combines real-time traversability estimation with classical local planning. By decoupling semantic reasoning from motion execution, PAPanner achieves embodiment-agnostic deployment and enables safe navigation without retraining. Extensive experiments on R2R-CE and RxR-CE benchmarks, as well as real-world deployments on humanoid, quadruped, and wheeled robots, demonstrate that PAPanner achieves superior generalization in unseen environments and robust sim-to-real transfer. Code and demonstration videos are available at: <https://ritonli.github.io/PAPanner/>.

Note to Practitioners—This research aims to improve the reliability of instruction execution progress estimation when applying VLN to real robots under long-horizon instructions. The proposed method organizes long instructions into ordered navigation stages and explicitly tracks the current execution progress to select actions within a structured command space. For motion execution, our method avoids robot-specific controllers and instead converts forward commands into safe local motions through LiDAR traversability estimation and classical local planning. Our method has been demonstrated through experiments on VLN-CE benchmarks and in real-world scenarios.

Index Terms—Robot navigation, vision-language model, embodiment-agnostic

I. INTRODUCTION

The application of vision-language models (VLMs) in robotic systems has become a key research area within embodied artificial intelligence, particularly in vision-and-language navigation in continuous environments (VLN-CE). Prior work in VLN-CE was largely grounded in discrete action spaces and graph-based representations, which restricted robotic agents to fixed environmental maps and predefined trajectories [1]–[6]. Within these architectures, the real environments are abstracted as connectivity graphs, and the navigation is cast as the teleportation over a waypoint set on such graphs [7]. Although these approaches have evolved rapidly and achieved

impressive performance [8]–[10], the reliance on discretized representations introduces practical limitations, such as difficulties in generalizing to new unstructured environments and mapping inaccuracies in dynamic scenes. The emergence of VLMs extends these foundational methods by enabling agents to directly process real-time egocentric visual observations alongside complex natural language instructions, acting as semantic reasoners to dynamically generate local waypoints or step-by-step action sequences without relying on pre-built environmental maps [11]–[15].

Despite these advances, VLM-based navigation policies still expose a critical bottleneck in long-horizon reasoning. When following lengthy and multi-stage instructions, existing VLM-based methods [13], [14], [16] typically condition each decision on the full instruction and recent observations, while the execution progress is implicitly maintained in the internal context of the VLM. Such implicit progress modeling is often insufficient for extended trajectories, which require the agent to determine which part of the instruction has been completed, which sub-instruction is currently active, and when to transition to the next sub-instruction. Without an explicit estimate of execution progress, the VLM may attend to completed or future sub-instructions, leading to unstable action prediction or premature stopping before reaching the target.

Aiming at the above issue, this work proposes PAPanner, a progress-aware planning framework for embodiment-agnostic vision-and-language navigation. At its core, we reformulate VLM-based navigation as a *progress-aware structured planning* problem. Instead of relying on implicit temporal reasoning, our high-level planner first uses a large language model (LLM) to decompose the instruction into an ordered set of sub-instructions and then explicitly tracks their execution progress while predicting navigation commands within a structured action space. This explicit progress tracking significantly enhances reasoning stability over long instructions and reduces errors caused by unreliable progress estimation. To translate these robust semantic decisions into physical motion while maintaining embodiment-agnostic deployment, the generated structured commands are processed by a low-level LiDAR-based locomotion policy. By integrating real-time traversability estimation with classical local planning, this policy ensures safe motion across diverse robotic platforms without requiring platform-specific learned policy training.

The main contributions of this work are summarized as follows:

- We propose PAPanner, a novel VLN framework that

maps complex language instructions into a structured action space, thus bridging the gap between semantic reasoning and physical execution.

- We formulate VLM-based navigation as progress-aware structured planning, where long-horizon instructions are decomposed into ordered sub-instructions and execution progress is explicitly tracked to stabilize decision-making over extended trajectories.
- Extensive experiments on R2R-CE and RxR-CE benchmarks, as well as real-world deployments on three different robotic platforms, demonstrate the superior generalization and robustness of our approach.

II. RELATED WORK

This section briefly reviews two research areas related to this work: robot navigation and vision-and-language navigation.

A. Robot Navigation Methods

Existing robot navigation approaches are divided into two main categories: classical methods and learning-based methods. Classical methods, including simultaneous localization and mapping (SLAM) and optimization-driven path planning techniques, have been extensively studied over the last two decades [17]–[19]. SLAM constructs spatial maps by collecting environmental data while simultaneously localizing the robot, thereby enabling autonomous navigation. These methods primarily rely on depth sensors and visual sensors, maintaining localization accuracy and path planning through continuous perception. Meanwhile, optimization-based motion planning methods excel in known scenarios, allowing for precise prediction of the robot future state and optimization of control inputs to generate optimal motion trajectories [20]–[22]. These algorithms are based on mathematical models, which search for optimal solutions within the control space and dynamically adjust the robot movement to ensure smooth obstacle avoidance. Although effective for traditional navigation tasks, these classical methods lack the ability to integrate vision-language features, making them insufficient for human-collaboration tasks such as vision-language navigation.

Data-driven learning models have been widely used to enhance robot adaptability [23]. Unlike classical methods, these models learn navigation behaviors directly from raw sensor data, allowing robots to generalize to more diverse environments [24]–[28]. Deep learning (DL) leverages neural networks to facilitate robots in learning end-to-end control strategies. Imitation learning (IL) enables robots to mimic expert demonstrations, thereby acquiring human-like motion patterns. Reinforcement learning (RL) trains robots to explore actions and receive feedback from environmental dynamics to maximize cumulative rewards. Additionally, Yang et al. [29] explored imperative learning (ImL), which integrates network prediction and metric-based optimization, forming a bi-level optimization process during training. However, these methods face challenges in real-world applications. DL, IL, and ImL are limited by the quality and diversity of their training data, the acquisition of which is often costly. RL models require extensive training and may not perform effectively due to

issues related to the transfer from simulated environments to real-world scenarios.

B. Vision-and-Language Navigation

VLN requires an embodied agent to interpret natural language instructions and navigate toward a specified target location based on visual observations [30]–[35]. Early VLN studies were conducted in discretized simulated environments, where the scene is represented as a connectivity graph and the agent selects its next viewpoint from a set of predefined neighboring nodes [8]–[10]. To better align visual observations with language instructions, Fried et al. [36] introduced the panoramic action space, in which the agent observes a panoramic view and chooses a direction corresponding to one of the candidate navigable viewpoints. This paradigm has enabled substantial progress in instruction following, cross-modal grounding, and long-horizon decision making. However, graph-based VLN abstracts navigation as transitions between discrete viewpoints, which differs significantly from real robotic navigation where an agent should move continuously and avoid obstacles. To bridge this gap, Krantz et al. [37] introduced VLN-CE, which transfers VLN from discrete navigation graphs to continuous 3D environments. Under this setting, the agent no longer teleports between graph nodes but executes low-level motion primitives such as moving forward and turning. Subsequent works have explored various ways to improve continuous navigation, including visual-language representations, memory modeling, waypoint prediction, and map-based planning [5], [6], [34], [35]. Although these methods [38]–[40] have improved VLN-CE performance, many of them depend on privileged inputs such as depth observations, panoramic views, odometry, or simulator-trained waypoint predictors.

VLMs [41]–[43] have provided a new direction for VLN by enabling agents to directly process egocentric visual observations and natural language instructions. Zhang et al. [7] introduced a video-based VLM planner that takes historical RGB observations as input and predicts the next navigation action. Cheng et al. [12] further proposed a vision-language-action framework that processes multi-frame observations and generates mid-level navigation commands in natural language, which are then executed by a learned locomotion policy. These VLM-based approaches reduce the reliance on explicit maps and handcrafted semantic representations, showing strong potential for open-world instruction following. However, most existing VLM-based methods still condition action prediction on the full instruction at each decision step, without explicitly modeling which part of the instruction has already been completed. They may suffer from stage confusion when handling long-horizon instructions with multiple landmarks, turns, and termination conditions. Another challenge lies in translating high-level semantic decisions into safe physical motion. Some methods directly execute VLM-predicted actions or use learned motion policies to bridge high-level reasoning and low-level control [12]. While effective on specific platforms, learned locomotion policies are often developed under specific robot morphologies, sensor configurations, and simulation environments. Transferring such policies to robots

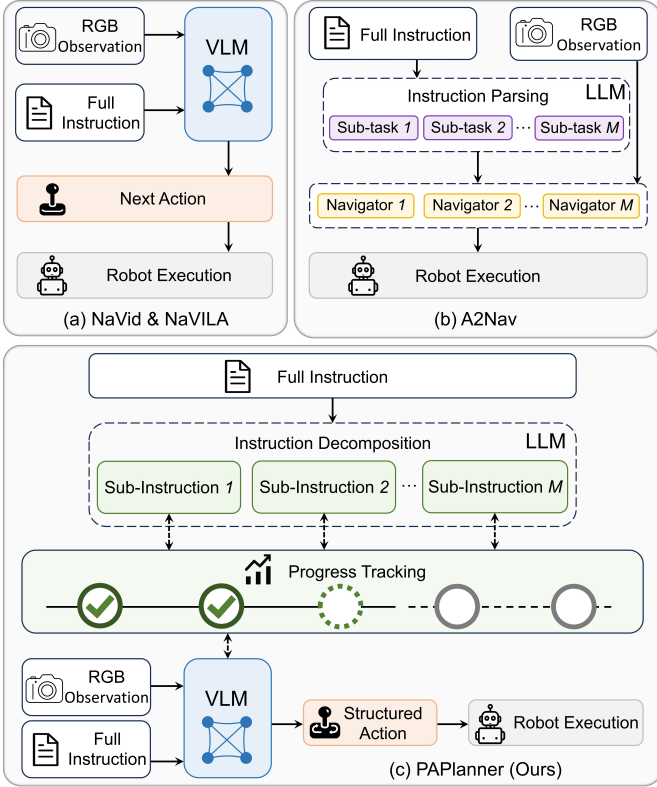


Fig. 1. Comparison of action generation architectures across different methods. (a) NaVILA and NaVid directly predict the next action from observations and the full instruction and execute it. (b) A2Nav parses the instruction into sequential sub-tasks and relies on learned navigators using a discrete action space. (c) PAPIanner decomposes the long-horizon instruction into ordered sub-instructions and explicitly tracks execution progress.

with different kinematics may require additional training or policy adaptation. In contrast, our work integrates classical navigation methods into a VLN framework, enabling smooth navigation without requiring the additional training.

Distinctions from Closely Related Work. Fig. 1 illustrates the differences between PAPIanner and the closely related methods. Specifically, recent VLM-based methods such as NaVid [7] and NaVILA [12] predict the next action from observations and the full instruction, offering limited explicit control over long-horizon execution progress and thus being prone to progress estimation errors. A2Nav [44] parses instructions into sequential sub-tasks but relies on learned navigators under a discrete navigation paradigm, which is cumbersome to scale and entails substantial engineering overhead. In contrast, PAPIanner decomposes long-horizon instructions into ordered sub-instructions and explicitly tracks execution progress, improving progress estimation and decision stability over long trajectories.

III. METHOD

This section elaborates PAPIanner, a progress-aware structured planning framework for embodiment-agnostic VLN. PAPIanner decouples semantic reasoning from platform-specific motion control by mapping long-horizon language instructions into structured navigation commands and executing forward

commands through LiDAR-based local planning. We first introduce the overall architecture in Sec. III-A. We then present progress-aware structured planning in Sec. III-B, including sub-instruction progress tracking, structured action prediction, and LiDAR-based locomotion policy. Finally, we describe the training and inference details of PAPIanner in Sec. III-C.

A. Overall Architecture

Fig. 2 illustrates the overall architecture of PAPIanner, which unifies progress-aware structured planning and LiDAR-based locomotion within a planning framework.

At the high level, we employ Qwen3-VL [45] as the visual-language backbone. The current RGB observation together with a sequence of historical frames is encoded into multi-frame visual tokens, which are aligned with the language token space through downsampling and projection. The navigation instruction is tokenized and fused with the visual tokens to form a joint visual-language representation. Rather than decoding this representation into free-form text, PAPIanner introduces a two-stage reasoning pipeline: *sub-instruction progress tracking* to identify the currently active task segment, followed by *structured action prediction* to output a standardized command from a structured action space.

At the low level, the locomotion policy consists of a LiDAR-based perception module that estimates the traversable area in real time and a classical local planner. When receiving a structured Forward command with a specific distance from the high level, the local planner generates a robot-centric target along the forward axis and refines it into a safe, short-horizon trajectory. Non-forward commands (e.g., turning or stopping) are executed directly by the robot controller.

B. Progress-aware Structured Planning

The proposed high-level planner reformulates VLM-based navigation as a progress-aware structured planning problem. Instead of asking the VLM to generate an action at every step, we explicitly model *which part of the instruction is currently relevant*, *which high-level action should be executed*, and *how far to advance or through what angle to turn when forward motion or turning is required*.

1) *Sub-instruction Progress Tracking*: While task decomposition has been widely explored in static robotic manipulation, applying it to VLN-CE presents a key challenge. Long-horizon VLN instructions often contain multiple stages, such as leaving a room, turning at a landmark, and finally stopping at a target object. Conditioning on the entire instruction at every step may cause weak progress awareness. To address this challenge, we decompose the instruction I into an ordered set of sub-instructions

$$\mathcal{S} = \{s_1, s_2, \dots, s_M\} \quad (1)$$

where each s_i denotes a semantically coherent and executable sub-instruction. We leverage the zero-shot reasoning capabilities of a Large Language Model to automatically parse the full instruction. Specifically, GPT-4o [46] is prompted to decompose a complex instruction into a sequence of sub-instructions under two strict constraints. Each sub-instruction

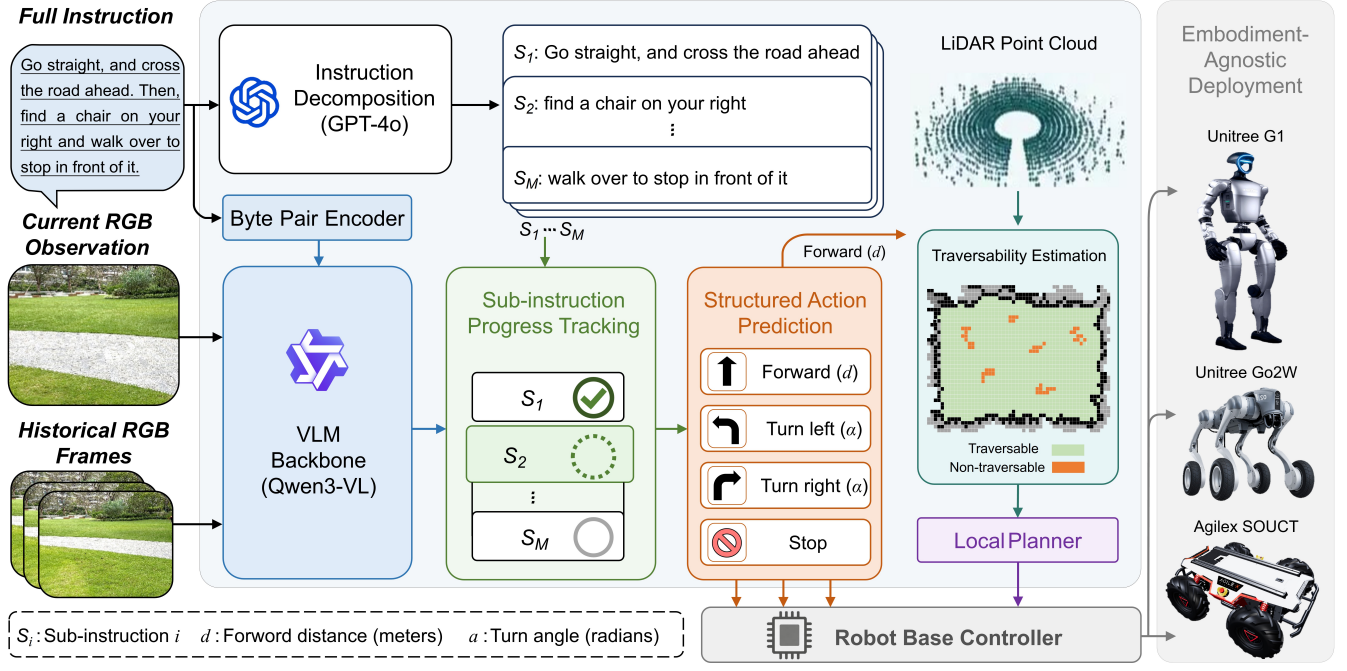


Fig. 2. Overview of PAPanner. At the high level, PAPanner first uses an LLM (GPT-4o) to parse the full instruction into an ordered set of sub-instructions, and then leverages a VLM (Qwen3-VL) to process multi-modal inputs, explicitly track sub-instruction progress, and predict structured navigation commands. At the low level, *Forward* commands are converted into robot-centric local goals and refined into safe physical motions through LiDAR-based traversability estimation and classical local planning, while *Turn left*, *Turn right*, and *Stop* commands are executed directly by the robot controller.

must contain a clear action verb (e.g., walk, turn, stop), and each sub-instruction must specify a spatial landmark or termination condition (e.g., at the end of the corridor, until you see a red bucket). For compound clauses (e.g., go past the sofa, turn near the table, and stop by the door), the LLM is required to split them into multiple ordered atomic steps. This decomposition determines the semantic order of the sub-instructions, while the timestep-level boundaries are further determined by trajectory alignment during training.

At timestep t , Qwen3-VL takes the current observation, historical frames, and navigation instruction as input. We use the final layer hidden state of the end of sequence (EOS) token as the joint visual-language representation

$$\mathbf{h}_t = \mathbf{H}_{\text{EOS}}^{\text{last}}. \quad (2)$$

Before navigation starts, each decomposed sub-instruction s_i is encoded by a frozen text encoder E_{text} . The token hidden states of s_i are mean-pooled and projected into the planner space

$$\mathbf{e}_i = W_e \cdot \text{MeanPool}(E_{\text{text}}(s_i)). \quad (3)$$

The sub-instruction embeddings are computed once after decomposition and reused during navigation.

To handle a variable number of sub-instructions, we predict progress by matching the current state with each valid sub-instruction

$$r_{t,i} = \frac{(W_q \mathbf{h}_t)^\top (W_k \mathbf{e}_i)}{\sqrt{d}}, \quad p_\theta(i | \mathbf{h}_t, \mathcal{S}) = \frac{\exp(r_{t,i})}{\sum_{j \in \mathcal{V}} \exp(r_{t,j})} \quad (4)$$

where $\mathcal{V}$ denotes the valid sub-instructions after padding, W_e , W_q , and W_k are learnable projection matrices, and d denotes

the feature dimension used for scaled dot-product matching. The one-step progress prediction is

$$z_t = \arg \max_{i \in \mathcal{V}} p_\theta(i | \mathbf{h}_t, \mathcal{S}). \quad (5)$$

The distribution p_θ is supervised by the aligned progress label y_t^{prog} . We use $\tilde{z}_t$ to denote the conditioning sub-instruction for downstream heads. During training, $\tilde{z}_t$ is set to the aligned label y_t^{prog} to keep the action supervision consistent with the sub-instruction context. During inference, the raw prediction is temporally filtered into $\hat{z}_t$, which can stay at the current sub-instruction or move to the next one, and we set $\tilde{z}_t = \hat{z}_t$. The completion score is predicted as

$$g_t = \sigma(\text{MLP}_g([\mathbf{h}_t; \mathbf{e}_{\tilde{z}_t}])) \quad (6)$$

where $\sigma(\cdot)$ denotes the sigmoid function, and g_t is used as an auxiliary training objective.

2) *Structured Action Prediction*: After identifying the relevant sub-instruction, the planner predicts an action type from $\{\text{Forward}, \text{Turn left}, \text{Turn right}, \text{Stop}\}$

$$a_t^{\text{type}} = \arg \max_c \text{MLP}_{\text{type}}([\mathbf{h}_t; \mathbf{e}_{\tilde{z}_t}])_c \quad (7)$$

where c indexes the action type. For *Forward*, the planner selects a distance from $\mathcal{D} = \{d^1, d^2, \dots, d^K\}$. For *Turn left* and *Turn right*, it selects an angle from $\mathcal{A} = \{\alpha^1, \alpha^2, \dots, \alpha^L\}$. The corresponding logits are

$$\mathbf{s}_t = \text{MLP}_d([\mathbf{h}_t; \mathbf{e}_{\tilde{z}_t}]), \quad \mathbf{u}_t = \text{MLP}_a([\mathbf{h}_t; \mathbf{e}_{\tilde{z}_t}]). \quad (8)$$

The selected distance and angle are

$$d_t^* = d^{\arg \max_j s_t^j}, \quad \alpha_t^* = \alpha^{\arg \max_\ell u_t^\ell} \quad (9)$$

where j and ℓ index distance and angle candidates, respectively. The navigation command is defined as

$$a_t = \begin{cases} \text{Forward}(d_t^*), & \text{if } a_t^{\text{type}} = \text{Forward} \\ \text{Turn left}(\alpha_t^*), & \text{if } a_t^{\text{type}} = \text{Turn left} \\ \text{Turn right}(\alpha_t^*), & \text{if } a_t^{\text{type}} = \text{Turn right} \\ \text{Stop}, & \text{if } a_t^{\text{type}} = \text{Stop}. \end{cases} \quad (10)$$

Compared with direct free-form action generation, the proposed formulation converts high-level navigation into structured decision making over a structured action space, leading to stable action prediction in VLN-CE.

3) *LiDAR-based Locomotion Policy*: As introduced in Sec. III-A, when the selected navigation command corresponds to a forward motion, it is passed to the LiDAR-based locomotion policy. The perception module in this stage adopts CADNet [47], which processes LiDAR point clouds to estimate traversable areas for safe motion execution. To train CADNet, we collected 4,500 raw point clouds at 5 Hz via remote control. Once trained, the weights of CADNet are frozen and deployed across all robotic platforms without any platform-specific fine-tuning. To evaluate the generalization capability of PAPIanner, the test environments are designed to be substantially different from those used for CADNet training.

For motion execution, when the high-level planner outputs $\text{Forward}(d_t^*)$, a robot-centric local target is generated on the robot’s forward axis

$$\mathbf{p}_t^{\text{goal}} = (d_t^*, 0) \quad (11)$$

where the local coordinate frame is centered at the robot base and the positive x -axis points forward. The estimated traversable region D is converted into a local cost map for TEB [20], which optimizes a short-horizon trajectory toward $\mathbf{p}_t^{\text{goal}}$ while respecting obstacle constraints and robot kinematic limits. The same CADNet weights are shared across all platforms, while platform-specific geometric and dynamic parameters are configured in the local planner.

C. Taming PAPIanner for Vision-and-Language Navigation

This work employs Qwen3-VL as the vision-language backbone of the VLN agent. We adapt it to navigation by introducing progress-aware reasoning and structured action prediction.

1) *Navigation Prompts*: In VLN, navigation prompts play a key role in guiding the model to interpret visual observations across different time steps. The image at the current timestep t represents the agent’s instantaneous perception of the environment and serves as the primary cue for action selection, while images from earlier timesteps form a sequence of historical observations that provide long-term contextual information. Therefore, we design language-based navigation prompts for Qwen3-VL, explicitly organizing and differentiating visual observations through natural language. In the navigation prompt, historical frames are described as the historical observation video to indicate their function in memory and progress tracking, whereas the most recent frame is referred to as the current observation,

highlighting its importance for real-time decision making. This prompt design enables Qwen3-VL to jointly reason over multi-frame visual inputs within a unified language-centric framework, integrating long-term navigation context with current environmental observation.

2) *Supervised Fine-tuning Data Blend*: Since the proposed high-level planner predicts both instruction progress and structured navigation commands, supervised fine-tuning (SFT) data should support both progress-aware reasoning and action prediction. Our SFT data blend is designed from three perspectives.

General VQA datasets. To enhance the general multi-modal understanding and instruction following ability of the model, we include general vision-language datasets, including ShareGPT4V and Video-ChatGPT.

Navigation data from simulation. To learn structured navigation decisions, we collect trajectories from R2R-CE and RxR-CE, including visual observations, robot poses, and expert actions. The LLM is used to decompose each instruction into ordered sub-instructions. Timestep-level progress labels are obtained by aligning these sub-instructions with continuous expert trajectory segments under a monotonicity constraint, using motion cues such as forward motion, turning points, and stopping positions. Timesteps within the segment assigned to s_i are labeled as $y_t^{\text{prog}} = i$. The completion label y_t^{comp} is computed as normalized path-length progress for forward segments or normalized yaw-angle progress for turning segments. Action-type labels are obtained from expert motion primitives, while distance and angle labels are generated by snapping expert displacement and yaw change to the nearest candidates in $\mathcal{D}$ and $\mathcal{A}$. Adjacent forward actions with similar motion patterns are aggregated to reduce action redundancy.

Navigation data from the real world. We collect 1.2k videos using onboard cameras mounted on three different mobile robots in a variety of indoor and outdoor environments. These environments exhibit different layouts, lighting conditions, and scene dynamics, which helps improve the robustness and generalization ability of the learned high-level planner.

Both simulated and real-world trajectories are processed using a unified frame sampling strategy. For each trajectory, we first obtain coarse navigation descriptions using a VLM and then refine them with an LLM to generate more diverse and natural navigation instructions.

3) *Training*: We fine-tune a pretrained Qwen3-VL backbone using the data described in Sec. III-C2. To support structured planning, we attach five lightweight task heads: progress prediction, completion scoring, action type prediction, forward distance prediction, and turn angle prediction. The training objective depends on the sample type. For general VQA samples, we apply the language modeling loss $\mathcal{L}_{\text{lm}}$. For navigation samples, we apply the navigation-specific losses

$$\mathcal{L}_{\text{nav}} = \mathcal{L}_{\text{prog}} + \lambda_1 \mathcal{L}_{\text{comp}} + \lambda_2 \mathcal{L}_{\text{type}} + \lambda_3 \mathcal{L}_{\text{dist}} + \lambda_4 \mathcal{L}_{\text{angle}} \quad (12)$$

where $\mathcal{L}_{\text{prog}}$ and $\mathcal{L}_{\text{type}}$ are cross-entropy losses over y_t^{prog} and y_t^{type} , while $\mathcal{L}_{\text{comp}}$ is the squared error between g_t and y_t^{comp} . $\mathcal{L}_{\text{dist}}$ is applied for **Forward**, while $\mathcal{L}_{\text{angle}}$ is applied for **Turn left** and **Turn right**. During supervised training,

the action heads are conditioned on y_t^{prog} to avoid inconsistent supervision.

4) *Inference*: During inference, GPT-4o first decomposes the instruction into ordered sub-instructions. Their embeddings are computed once by the text encoder and reused throughout the episode. At each timestep, Qwen3-VL encodes the current observation, historical frames, and instruction, and the final layer hidden state of the EOS token is used as $\mathbf{h}_t$. The progress tracker predicts $p_\theta(i | \mathbf{h}_t, \mathcal{S})$, and the raw prediction is temporally filtered to obtain $\hat{z}_t$. We set $\tilde{z}_t = \hat{z}_t$ and use it to condition all downstream heads for action-type, distance, and angle prediction. The resulting structured command is then sent to the corresponding execution module.

IV. EXPERIMENTS

In this section, we demonstrate the superiority of PAPIanner in both simulated and real-world environments via extensive comparisons.

A. Experimental Setups

1) *Simulated Environments*: Our experiments focus on two widely used VLN-CE benchmarks, R2R-CE [48] and RxR-CE [49]. To ensure a fair comparison, we adopt specific training data configurations for different evaluation protocols. The models used in our experiments (including real-world deployments and ablation studies) are trained on the combined training splits of both R2R-CE and RxR-CE. However, for the evaluation of cross-dataset generalization on the RxR-CE benchmark, the model is trained exclusively on the R2R-CE train split without exposing any RxR-CE samples during training.

2) *Real-World Environments*: To evaluate the performance of PAPIanner in real-world environments, we select three indoor scenes, including conference rooms, laboratories, and corridors, as well as three outdoor scenes, including campus roads, parking lots, and gardens. Regarding navigation instructions, we consider two types of instructions: simple landmark instructions, which consist of basic movements, and complex compositional instructions, which require the robot to navigate toward specific objects or locations. All real-world experiments are conducted on three different robotic platforms, including the Unitree G1 humanoid robot, the Unitree Go2W quadruped robot, and the AgileX Scout unmanned ground vehicle.

3) *Metrics*: We evaluate our method using four standard metrics commonly adopted in VLN tasks:

- **Success rate (SR)**. The percentage of navigation episodes in which the robot successfully stops within three meters radius of the target location.
- **Navigation error (NE)**. The average distance between the robot’s final stopping position and the target viewpoint.
- **Success weighted by path length (SPL)**. The success rate normalized by the ratio between the length of the shortest path and the predicted path.
- **Trajectory length (TL)**. The average length of the navigation trajectories executed by the robot.

TABLE I
COMPARISON WITH DIFFERENT METHODS IN TERMS OF TL, NE, SR, AND SPL ON THE R2R-CE VAL UNSEEN SPLIT. * INDICATES METHODS USING THE WAYPOINT PREDICTOR FROM HONG ET AL. [5]

Methods	Observation			Val Unseen			
	S.RGB	Pano.	Depth Odo.	TL	NE↓	SR↑	SPL↑
VLN○BERT* [5]	✓	✓	✓	12.23	5.74	44.0	39.0
GridMM* [6]	✓	✓	✓	13.36	5.11	49.0	41.0
Ego2-Map* [50]	✓	✓	✓	-	5.54	47.0	41.0
DreamWalker* [51]	✓	✓	✓	-	5.53	49.0	44.0
ETPNav* [52]	✓	✓	✓	11.99	4.71	57.0	49.0
HNR* [53]	✓	✓	✓	-	4.42	61.0	51.0
HAMT* [54]	✓	✓	✓	11.46	4.80	55.0	51.0
BEVBert* [55]	✓	✓	✓	-	4.57	59.0	50.0
AG-CMTP [30]	✓	✓	✓	-	7.90	23.1	19.1
R2R-CMTP [30]	✓	✓	✓	-	7.90	26.4	22.7
LAW [31]	✓	✓	✓	8.89	6.83	35.0	31.0
CM2 [32]	✓	✓	✓	11.54	7.02	34.3	27.6
WS-MGMap [33]	✓	✓	✓	10.00	6.28	38.9	34.3
AO-Planner [34]	✓	✓	✓	-	5.55	47.0	33.0
Seq2Seq [35]	✓	✓	✓	9.30	7.77	25.0	22.0
CMA [35]	✓	✓	✓	8.64	7.37	32.0	30.0
RGB-Seq2Seq [35]	✓			4.86	10.10	0.0	0.0
RGB-CMA [35]	✓			6.28	9.55	5.0	4.4
NaVid [7]	✓			7.63	5.47	37.4	35.9
NaVILA [12]	✓			-	<u>5.22</u>	<u>54.0</u>	<u>49.0</u>
PAPIanner (Ours)	✓			8.10	5.17	54.7	49.4

TABLE II
CROSS-DATASET PERFORMANCE IN TERMS OF TL, NE, SR, AND SPL ON THE RxR-CE VAL UNSEEN SPLIT. ALL RESULTS ARE OBTAINED WITHOUT TRAINING ON THE RxR-CE TRAINING SET.

Methods	Observation			Val Unseen			
	S.RGB	Depth	Odo.	TL	NE↓	SR↑	SPL↑
LAW [31]	✓	✓	✓	4.01	10.87	8.0	8.0
CM2 [32]	✓	✓	✓	12.29	8.98	14.4	9.2
WS-MGMap [33]	✓	✓	✓	10.80	9.83	15.0	12.1
VLN○BERT [5]	✓	✓	✓	16.78	-	30.6	24.9
Seq2Seq [35]	✓	✓		1.16	11.80	3.5	3.4
CMA [35]	✓	✓		5.09	11.70	4.4	2.4
RGB-Seq2Seq [35]	✓			4.43	11.20	0.0	0.0
RGB-CMA [35]	✓			13.56	9.55	0.0	0.0
A2Nav [44]	✓			-	-	16.8	6.3
NaVid [7]	✓			10.59	8.41	23.8	21.2
NaVILA [12]	✓			-	8.78	<u>34.3</u>	<u>28.2</u>
PAPIanner (Ours)	✓			11.03	<u>8.53</u>	34.6	28.7

B. Evaluation in Simulation

1) *R2R-CE Benchmark*: We first conduct experiments on the R2R-CE Val-Unseen split to evaluate the cross-split generalizability of different methods, with the results listed in Table I. Our method only uses RGB observations. Compared with related waypoint-prediction-based approaches, our method achieves comparable performance in terms of NE, SR, and SPL, despite not using depth or odometry data. Under the same RGB-only setting, our method outperforms all baselines in terms of NE, SR and SPL.

TABLE III
RESULTS OF USING DIFFERENT TRAINING DATASET COMPOSITIONS ON THE R2R-CE VAL UNSEEN SPLIT.

Training Data	Val-Unseen			
	TL	NE↓	SR↑	SPL↑
Sim.	7.52	5.79	48.0	38.9
Sim.+VQA	9.06	5.31	52.6	45.8
Sim.+VQA+Real.	8.10	5.17	54.7	49.4

2) *RxR-CE Benchmark*: Compared with R2R-CE, RxR-CE poses greater challenges in terms of instruction complexity, trajectory length, and language diversity. To evaluate cross-dataset generalization, we follow [7], [12] and train PAPIanner exclusively on the R2R-CE dataset, without using any RxR-CE training samples. As shown in Table II, our method achieves the best SR and SPL among all compared approaches, reaching 34.6% and 28.7%, respectively, while remaining competitive on NE.

3) *Impact of different training data composition*: We carry out experiments to analyze the impact of different training data compositions, using three data configurations: navigation samples from the VLN-CE simulation dataset, vision-language dataset, and real-world navigation behavior samples. As shown in Table III, based on the simulation dataset, incorporating vision-language datasets yielded the most significant gains, which can be attributed to the fact that large-scale vision-language data provide richer visual-semantic alignments and enhance the model’s ability to understand diverse instructions and visual scenes. Our final configuration further incorporates real-world navigation behaviors via self-collected data, enabling PAPIanner to achieve better performance. These results highlight that increasing data diversity substantially improves model performance.

4) *Different Number of Historical Frames*: To evaluate the influence of temporal visual context on progress-aware planning, we conducted an ablation study by varying the number of historical frames. In this experiment, we use simulated navigation data from the VLN-CE datasets for training, without incorporating general VQA data or real-world navigation data. The model architecture and training protocol were kept unchanged, while the number of historical frames was set to 4, 8, and 16, respectively. As listed in Table IV, using 4 historical frames resulted in limited temporal context, which was insufficient for reliably identifying visited regions and sub-instruction transitions. Increasing the history length to 8 frames improved navigation performance, suggesting that richer temporal observations facilitate more accurate progress estimation and more stable action prediction. Although using 16 historical frames provided a longer observation history, it also increased the visual token budget and introduced redundant observations. Therefore, we use 8 historical frames as the default setting, as it provides sufficient temporal context for progress-aware planning while avoiding unnecessary input redundancy.

5) *Ablation on Sub-instruction Progress Tracking*: To verify the effectiveness of the sub-instruction progress tracking

TABLE IV
EFFECT OF THE NUMBER OF HISTORICAL FRAMES ON THE R2R-CE VAL UNSEEN SPLIT.

History Frames	TL	NE↓	SR↑	SPL↑
4	10.26	5.89	43.6	33.9
8	7.52	5.79	48.0	38.9
16	8.17	5.94	48.2	36.2

TABLE V
ABLATION STUDY OF SUB-INSTRUCTION PROGRESS TRACKING ON THE R2R-CE VAL UNSEEN SPLIT.

Method	TL	NE↓	SR↑	SPL↑
PAPIanner (w/o Progress Tracking)	8.68	5.42	52.8	46.1
PAPIanner	8.10	5.17	54.7	49.4

mechanism, we conduct an ablation study by removing the progress prediction branch from PAPIanner. In this variant, the planner predicts structured navigation commands from the joint visual-language representation conditioned on the full instruction, without explicitly selecting the currently relevant sub-instruction. As shown in Table V, removing sub-instruction progress tracking degrades the navigation performance. This indicates that conditioning action prediction on the current sub-instruction helps the model focus on the appropriate navigation stage and improves path efficiency in long-horizon instruction following.

C. Evaluation in Real World

1) *Comparison in Different Environments*: We conduct extensive real-world experiments across 6 environments to assess the generalization capability of our method. Based on the results of the simulation experiments in the previous section, we select two state-of-the-art methods, NaVid and NaVILA, for comparison. Our method is compared with NaVid on a Scout, and with NaVILA on a Go2W. Each method is evaluated in the different environments described in Sec. IV-A2, with two levels of instruction difficulty. For each environment, we design 25 simple instructions and 25 complex instructions, and each instruction is executed twice. The results are presented in Table VI. Our method consistently outperforms both competing methods in terms of SR and NE. This advantage mainly stems from decoupling semantic reasoning from motion execution. The progress-aware structured planning enables more accurate instruction following by explicitly tracking the relevant sub-instruction, while the LiDAR-based locomotion policy improves the robustness of forward motion execution in real-world environments.

We further report the average per-step inference time to evaluate real-time behavior under identical deployment conditions. NaVid exhibits the highest inference time, which we attribute to the substantial visual token budget resulting from its dense stacking of historical frames. While NaVILA reduces this cost by using a fixed 8-frame historical buffer, it still generates mid-level navigation commands in natural language through autoregressive LLM decoding and then parses the

TABLE VI
REAL-WORLD COMPARISON ON LEGGED AND WHEELED ROBOTS IN 6 ENVIRONMENTS. **SIMP. I** AND **COMP. I** REFER TO SIMPLE AND COMPLEX INSTRUCTION-FOLLOWING TASKS, RESPECTIVELY. **AVG. T.** DENOTES THE AVERAGE PER-STEP MODEL INFERENCE TIME (S).

Methods	Conference Room		Laboratory		Corridor		Campus Road		Parking Lot		Garden		Avg. T.												
	Simp. I.	Comp. I.	Simp. I.	Comp. I.	Simp. I.	Comp. I.	Simp. I.	Comp. I.	Simp. I.	Comp. I.	Simp. I.	Comp. I.													
	SR↑	NE↓	SR↑	NE↓	SR↑	NE↓	SR↑	NE↓	SR↑	NE↓	SR↑	NE↓													
NaVid [7]	-	-	-	-	-	-	88	0.63	48	0.99	62	1.24	44	1.96	52	2.32	18	3.51	8	3.49	0	5.07	1.89		
PAPlanner (Ours)	-	-	-	-	-	-	92	0.58	64	0.83	86	0.97	64	1.22	74	1.15	48	1.83	30	2.62	20	2.90	1.16		
NaVILA [12]	84	0.63	56	0.98	86	0.67	50	1.20	90	0.64	52	0.93	76	1.03	46	2.18	60	2.09	26	2.40	20	3.37	4	4.94	1.34
PAPlanner (Ours)	92	0.60	68	0.91	90	0.53	60	0.95	92	0.55	66	0.79	88	0.91	64	1.03	74	1.10	50	1.77	32	2.09	24	2.86	1.16



Go straight, turn left at the black fan to go out through the door, and stop as soon as you find a cardboard box.



Walk forward and bypass the car ahead from its right side. Then, locate a yellow bicycle and stop next to it.



Go straight to cross the road ahead, and re-enter the lawn area. Then, find a chair on your right and walk over to stop in front of it.

Fig. 3. Real-world deployment of PAPLanner. Given a lengthy instruction, different types of robots are able to correctly parse the instruction and stop at the specified destination.

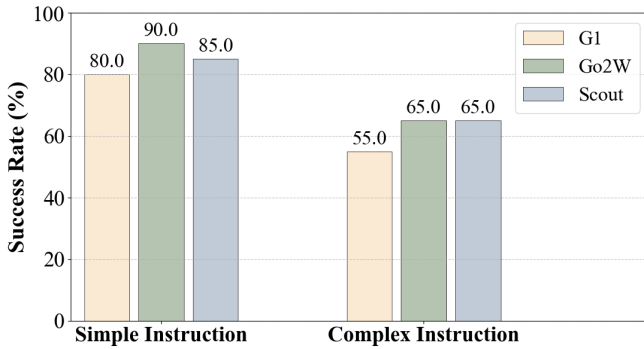


Fig. 4. Success rates of three different robots for simple and complex instructions, respectively.

generated text into action types and arguments. In contrast, PAPLanner uses the same fixed 8-frame historical buffer but predicts structured navigation commands directly from the EOS representation using lightweight task heads. This avoids iterative text generation and reduces decoding overhead, leading to the lowest average per-step inference time.

2) *Embodiment-Agnostic Evaluation:* To evaluate embodiment-agnostic the deployment of PAPLanner, we conduct experiments on three different robotic platforms (G1, Go2W, and Scout). Using the same trained model, we perform 40 trials on each platform, including 10 simple instructions and 10 complex instructions, with each instruction executed twice. Fig. 3 presents representative examples of real-world navigation achieved by different robot platforms using our framework. As shown in Fig. 4, navigation performance remains relatively consistent across platforms, indicating that PAPLanner generalizes well to robots with different locomotion types. It is noted that the success rate of PAPLanner on the G1 platform is slightly lower than that on the other robots. This is expected due to the larger physical size and limited maneuverability of the humanoid robot, which makes navigation in cluttered environments more challenging.

3) *Low-level Locomotion Policy Performance:* To highlight the advantages of our locomotion policy, we compare it against NaVILA and an additional RL-based baseline. Specifically, the RL-based motion policy is trained using the perception outputs

TABLE VII
PERFORMANCE OF LOW-LEVEL POLICY IN TERMS OF AVERAGE
COMPLETION TIME AND COLLISION RATE.

Method	Average Completion Time	Collision Rate
NaVILA [12]	59.37	6/25
Reinforcement Learning	51.54	5/25
PAPanner (Ours)	48.92	2/25

of CADNet as observations. All experiments are conducted in real-world indoor environments on a Go2W under identical hardware settings. We evaluate these motion methods in terms of average completion time and collision rate. The average completion time measures the efficiency with which the robot successfully completes an instruction, while the collision rate quantifies navigation safety in terms of physical contact with the environment. We conduct a total of 25 trials using simple instructions, and the quantitative results of all three methods are shown in Table VII. Across the 25 trials, our method results in two minor collisions, whereas the baseline methods exhibit more frequent collisions and longer completion times. The main reason is that our locomotion policy is more robust to environmental uncertainty in real-world scenarios. In contrast, the RL-based baseline in our implementation is sensitive to perception noise and the distribution shift between training and deployment, which can lead to state deviations and reduced policy stability.

V. CONCLUSION

This paper presents PAPanner, a progress-aware planning framework designed for embodiment-agnostic VLN. Our work demonstrates that reformulating VLM-based navigation as a structured decision-making problem effectively bridges the gap between open-ended semantic reasoning and safe physical execution. By explicitly tracking sub-instruction progress and predicting navigation commands within a structured action space, PAPanner reduces errors caused by unreliable estimation of instruction execution progress over long-horizon trajectories. Extensive experiments across simulation benchmarks and diverse real-world scenarios demonstrate that PAPanner achieves superior generalization in unseen environments and robust sim-to-real transfer. In the future work, we plan to develop a VLM-driven visual feedback mechanism to actively detect execution failures and autonomously replan sub-instructions when facing unexpected environmental changes.

REFERENCES

- [1] D. An, H. Wang, W. Wang, Z. Wang, Y. Huang, K. He, and L. Wang, "Etpnav: Evolving topological planning for vision-language navigation in continuous environments," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.
- [2] J. Krantz, S. Banerjee, W. Zhu, J. Corso, P. Anderson, S. Lee, and J. Thomason, "Iterative vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 14 921–14 930.
- [3] Y.-H. H. Tsai, V. Dhar, J. Li, B. Zhang, and J. Zhang, "Multimodal large language model for visual navigation," *arXiv preprint arXiv:2310.08669*, 2023.
- [4] Z. Wang, J. Li, Y. Hong, Y. Wang, Q. Wu, M. Bansal, S. Gould, H. Tan, and Y. Qiao, "Scaling data generation in vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 12 009–12 020.
- [5] Y. Hong, Z. Wang, Q. Wu, and S. Gould, "Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 15 439–15 449.
- [6] Z. Wang, X. Li, J. Yang, Y. Liu, and S. Jiang, "Gridmm: Grid memory map for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 15 625–15 636.
- [7] J. Zhang, K. Wang, R. Xu, G. Zhou, Y. Hong, X. Fang, Q. Wu, Z. Zhang, and H. Wang, "Navid: Video-based vlm plans the next step for vision-and-language navigation," *arXiv preprint arXiv:2402.15852*, 2024.
- [8] Y. Long, X. Li, W. Cai, and H. Dong, "Discuss before moving: Visual language navigation via multi-expert discussions," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. IEEE, 2024, pp. 17 380–17 387.
- [9] D. Shah, B. Osinski, S. Levine *et al.*, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Proc. Conf. Robot. Learn. (CoRL)*. pmlr, 2023, pp. 492–504.
- [10] G. Zhou, Y. Hong, and Q. Wu, "Navgpt: Explicit reasoning in vision-and-language navigation with large language models," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 7, 2024, pp. 7641–7649.
- [11] D. Zheng, S. Huang, Y. Li, and L. Wang, "Efficient-vln: A training-efficient vision-language navigation model," *arXiv preprint arXiv:2512.10310*, 2025.
- [12] A.-C. Cheng, Y. Ji, Z. Yang, Z. Gongye, X. Zou, J. Kautz, E. Bıyık, H. Yin, S. Liu, and X. Wang, "Navila: Legged robot vision-language-action model for navigation," *arXiv preprint arXiv:2412.04453*, 2024.
- [13] L. Zhang, X. Hao, Q. Xu, Q. Zhang, X. Zhang, P. Wang, J. Zhang, Z. Wang, S. Zhang, and R. Xu, "Mapnav: A novel memory representation via annotated semantic maps for vlm-based vision-and-language navigation," in *Proc. Annu. Meet. Assoc. Comput. Linguistics (ACL)*, 2025, pp. 13 032–13 056.
- [14] H. Ding, Z. Xu, Y. Fang, Y. Wu, Z. Chen, J. Shi, J. Huo, Y. Zhang, and Y. Gao, "Lavira: Language-vision-robot actions translation for zero-shot vision language navigation in continuous environments," *arXiv preprint arXiv:2510.19655*, 2025.
- [15] Z. Wang, S. Lee, and G. H. Lee, "Dynam3d: Dynamic layered 3d tokens empower vlm for vision-and-language navigation," *arXiv preprint arXiv:2505.11383*, 2025.
- [16] M. Elnoor, K. Weerakoon, G. Seneviratne, J. Liang, V. Rajagopal, and D. Manocha, "Vi-lad: Vision-language attention distillation for socially-aware robot navigation in dynamic environments," *arXiv preprint arXiv:2503.09820*, 2025.
- [17] S. Barlas, D. Alexiou, K. Tsiakas, D. Katsatos, I. Kostavelis, D. Giakoumis, A. Gasteratos, and D. Tzovaras, "Robot active vision-based path planning for localization improvement in indoor environments," in *Proc. IEEE/RISJ Int. Conf. Intell. Robots Syst. (IROS)*. IEEE, Oct. 2024, pp. 10 686–10 693.
- [18] J. Cao, S. Khan, W. Liu, Y. Li, and B. Vucetic, "Remote ugv control via practical wireless channels: A model predictive control approach," *IEEE Trans. Intell. Veh.*, pp. 1–13, 2024.
- [19] Y. Song, R. Wang, Q. Bi, Z. Pan, H. Gao, and X. Zhang, "Stvo: Spatial-temporal constrained velocity obstacle for safe navigation among pedestrians," *IEEE Trans. Veh. Technol.*, vol. 74, no. 9, pp. 13 580–13 591, 2025.
- [20] C. Rosmann, W. Feiten, T. Wosch, F. Hoffmann, and T. Bertram, "Trajectory modification considering dynamic constraints of autonomous robots," in *7th German Conference on Robotics*. VDE, 2012, pp. 1–6.
- [21] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 3067–3074.
- [22] W. Chi, J. Yu, C. Cao, and L. Sun, "Ptan: Probability-aware topological adaptive navigation for mobile robots in human-dense dynamic environments," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 7807–7818, 2026.
- [23] R. Li, Y. Zhao, X. Xu, J. Chen, R. Song, and W. Zhang, "Papnet: Point-enhanced attention-aware pillar network for 3d object detection in autonomous driving," *IEEE Transactions on Automation Science and Engineering*, 2026.
- [24] R. Li, Y. Xie, L. Zhang, T. Li, and W. Zhang, "A continual imitation learning benchmark for mobile robot navigation in sequential environments," *Robot. Intell. Autom.*, 2025.

- [25] R. Li, L. Zhang, Y. Zhao, C. Liu, R. Song, and W. Zhang, "Drl-th: Jointly utilizing temporal graph attention and hierarchical fusion for ugv navigation in crowded environments," *arXiv preprint arXiv:2512.24284*, 2025.
- [26] Y. Zhao, X. Xu, R. Li, R. Song, and W. Zhang, "Rakd: Risk-aware knowledge distillation for autonomous driving in adversarial scenarios," *IEEE Trans. Veh. Technol.*, 2025.
- [27] B. Cai, C. Wei, and Z. Ji, "Deep reinforcement learning with multiple unrelated rewards for agv mapless navigation," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 4323–4340, 2024.
- [28] Z. Wang and G. Tian, "Goal-oriented visual semantic navigation using semantic knowledge graph and transformer," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1647–1657, 2024.
- [29] P. Roth, J. Nubert, F. Yang, M. Mittal, and M. Hutter, "ViPlanner: Visual semantic imperative learning for local navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. IEEE, May. 2024, pp. 5243–5249.
- [30] K. Chen, J. K. Chen, J. Chuang, M. Vázquez, and S. Savarese, "Topological planning with transformers for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 11 276–11 286.
- [31] S. Raychaudhuri, S. Wani, S. Patel, U. Jain, and A. Chang, "Language-aligned waypoint (law) supervision for vision-and-language navigation in continuous environments," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2021, pp. 4018–4028.
- [32] G. Georgakis, K. Schmeckpeper, K. Wanchoo, S. Dan, E. Miltsakaki, D. Roth, and K. Daniilidis, "Cross-modal map learning for vision and language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 15 460–15 470.
- [33] P. Chen, D. Ji, K. Lin, R. Zeng, T. Li, M. Tan, and C. Gan, "Weakly-supervised multi-granularity map learning for vision-and-language navigation," *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, pp. 38 149–38 161, 2022.
- [34] J. Chen, B. Lin, X. Liu, L. Ma, X. Liang, and K.-Y. K. Wong, "Affordances-oriented planning using foundation models for continuous vision-language navigation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 22, 2025, pp. 23 568–23 576.
- [35] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. European Conf. Comput. Vis. (ECCV)*. Springer, 2020, pp. 104–120.
- [36] D. Fried, R. Hu, V. Cirik, A. Rohrbach, J. Andreas, L.-P. Morency, T. Berg-Kirkpatrick, K. Saenko, D. Klein, and T. Darrell, "Speaker-follower models for vision-and-language navigation," *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018.
- [37] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. European Conf. Comput. Vis. (ECCV)*. Springer, 2020, pp. 104–120.
- [38] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 537–16 547.
- [39] K. Chen, D. An, Y. Huang, R. Xu, Y. Su, Y. Ling, I. Reid, and L. Wang, "Constraint-aware zero-shot vision-language navigation in continuous environments," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [40] J. Krantz, A. Gokaslan, D. Batra, S. Lee, and O. Maksymets, "Waypoint models for instruction-guided navigation in continuous environments," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 15 162–15 171.
- [41] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-vl technical report," *arXiv preprint arXiv:2511.21631*, 2025.
- [42] W. Wang, Z. Gao, L. Gu, H. Pu, L. Cui, X. Wei, Z. Liu, L. Jing, S. Ye, J. Shao *et al.*, "Internvl3. 5: Advancing open-source multimodal models in versatility, reasoning, and efficiency," *arXiv preprint arXiv:2508.18265*, 2025.
- [43] W. Hong, W. Yu, X. Gu, G. Wang, G. Gan, H. Tang, J. Cheng, J. Qi, J. Ji, L. Pan *et al.*, "Glm-4.5 v and glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning," *arXiv preprint arXiv:2507.01006*, 2025.
- [44] P. Chen, X. Sun, H. Zhi, R. Zeng, T. H. Li, G. Liu, M. Tan, and C. Gan, "A2nav: Action-aware zero-shot robot navigation by exploiting vision-and-language ability of foundation models," *arXiv preprint arXiv:2308.07997*, 2023.
- [45] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-vl technical report," *arXiv preprint arXiv:2511.21631*, 2025.
- [46] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [47] S. Xie, R. Song, Y. Zhao, X. Huang, Y. Li, and W. Zhang, "Circular accessible depth: A robust traversability representation for ugv navigation," *IEEE Trans. Robot.*, pp. 4875–4891, Dec. 2023.
- [48] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. European Conf. Comput. Vis. (ECCV)*. Springer, 2020, pp. 104–120.
- [49] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, "Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2020, pp. 4392–4412.
- [50] Y. Hong, Y. Zhou, R. Zhang, F. Démoncourt, T. Bui, S. Gould, and H. Tan, "Learning navigational visual representations with semantic map supervision," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 3055–3067.
- [51] H. Wang, W. Liang, L. Van Gool, and W. Wang, "Dreamwalker: Mental planning for continuous vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 10 873–10 883.
- [52] D. An, H. Wang, W. Wang, Z. Wang, Y. Huang, K. He, and L. Wang, "Etpnav: Evolving topological planning for vision-language navigation in continuous environments," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.
- [53] Z. Wang, X. Li, J. Yang, Y. Liu, J. Hu, M. Jiang, and S. Jiang, "Lookahead exploration with neural radiance representation for continuous vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 13 753–13 762.
- [54] Z. Wang, J. Li, Y. Hong, Y. Wang, Q. Wu, M. Bansal, S. Gould, H. Tan, and Y. Qiao, "Scaling data generation in vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 12 009–12 020.
- [55] D. An, Y. Qi, Y. Li, Y. Huang, L. Wang, T. Tan, and J. Shao, "Bevbert: Multimodal map pre-training for language-guided navigation," *arXiv preprint arXiv:2212.04385*, 2022.